

```
///! Adashe: trustless rotating savings
circles (adashe / esusu / ajo / ROSCA) on
Solana.
///!
///! N members each contribute
`contribution` every round. Each round
exactly one member
///! (by join slot) receives the pot of  $(N-1) * \text{contribution}$ . The recipient does not
pay
///! into their own round, so every member's
total obligation is exactly  $(N-1) * \text{contribution}$ .
///! Each member locks that same amount as
collateral up front. If someone misses a
round,
///! anyone can call `cover_default` after
the deadline and their collateral pays
their share.
///!
///! SOLVENCY INVARIANT (see
docs/THREAT_MODEL.md):
///! vault_balance ==
sum(member.collateral) + (contributions
held for the current round)
///! and for every member: member.collateral
 $\geq (N-1 - \text{rounds\_settled\_by\_member}) * \text{contribution}$ .

use anchor_lang::prelude::*;
```

```

use anchor_spl::token::{self, Mint, Token,
TokenAccount, Transfer};

declare_id!
("Fg6PaFpoGXkYsidMpWTK6W2BeZ7FEfcYkg476zPFs
LnS");

pub const MIN_MEMBERS: u8 = 2;
pub const MAX_MEMBERS: u8 = 24;
pub const MIN_PERIOD_SECS: i64 = 3_600; //
1 hour
pub const MAX_PERIOD_SECS: i64 = 90 *
86_400; // 90 days
pub const MIN_JOIN_WINDOW_SECS: i64 =
3_600;
pub const MAX_JOIN_WINDOW_SECS: i64 = 30 *
86_400;

#[program]
pub mod adashe {
    use super::*;

    /// Create a circle and its vault. The
    creator is NOT automatically a member.
    pub fn create_circle(
        ctx: Context<CreateCircle>,
        circle_id: u64,
        contribution: u64,
        period_secs: i64,
        max_members: u8,
        join_window_secs: i64,
    ) -> Result<()> {
        require!(contribution > 0,
AdasheError::InvalidParams);

```

```

        require!(
            (MIN_MEMBERS..=MAX_MEMBERS).contains(&max_m
embers),
            AdasheError::InvalidParams
        );
        require!(
            (MIN_PERIOD_SECS..=MAX_PERIOD_SECS).contain
s(&period_secs),
            AdasheError::InvalidParams
        );
        require!(
            (MIN_JOIN_WINDOW_SECS..=MAX_JOIN_WINDOW_SEC
S).contains(&join_window_secs),
            AdasheError::InvalidParams
        );
        // Make sure pot/collateral math
can never overflow later.
        contribution
            .checked_mul((max_members - 1)
as u64)

        .ok_or(AdasheError::MathOverflow)?;

        let now =
Clock::get()?.unix_timestamp;
        let c = &mut ctx.accounts.circle;
        c.creator =
ctx.accounts.creator.key();
        c.mint = ctx.accounts.mint.key();
        c.vault = ctx.accounts.vault.key();
        c.circle_id = circle_id;

```

```

        c.contribution = contribution;
        c.period_secs = period_secs;
        c.join_deadline = now
            .checked_add(join_window_secs)

.ok_or(AdasheError::MathOverflow)?;
    c.round_deadline = 0;
    c.max_members = max_members;
    c.member_count = 0;
    c.round = 0;
    c.paid_count = 0;
    c.status = CircleStatus::Forming;
    c.bump = ctx.bumps.circle;

    emit!(CircleCreated {
        circle: c.key(),
        creator: c.creator,
        mint: c.mint,
        contribution,
        max_members,
        period_secs,
    });
    Ok(())
}

/// Join a forming circle and lock
collateral of (N-1) * contribution.
/// Join order decides payout order
(slot 0 is paid first).
pub fn join_circle(ctx:
Context<JoinCircle>) -> Result<()> {
    let now =
Clock::get()?.unix_timestamp;
    let circle = &mut

```

```

ctx.accounts.circle;
    require!(circle.status ==
CircleStatus::Forming,
AdasheError::WrongStatus);
    require!(now <=
circle.join_deadline,
AdasheError::JoinWindowClosed);
    require!(circle.member_count <
circle.max_members,
AdasheError::CircleFull);

    let collateral =
circle.pot_size()?;

    let member = &mut
ctx.accounts.member;
    member.circle = circle.key();
    member.authority =
ctx.accounts.authority.key();
    member.slot = circle.member_count;
    member.settled = 0;
    member.collateral = collateral;
    member.defaults = 0;
    member.bump = ctx.bumps.member;

    circle.member_count = circle
        .member_count
        .checked_add(1)

    .ok_or(AdasheError::MathOverflow)?;
    if circle.member_count ==
circle.max_members {
        circle.status =
CircleStatus::Active;

```

```

        circle.round = 0;
        circle.paid_count = 0;
        circle.round_deadline = now

.checked_add(circle.period_secs)

.ok_or(AdasheError::MathOverflow)?;
    }
    let slot = member.slot;
    let circle_key = circle.key();

    token::transfer(
        CpiContext::new(

ctx.accounts.token_program.to_account_info(
),
            Transfer {
                from:
ctx.accounts.authority_token.to_account_info(),
                to:
ctx.accounts.vault.to_account_info(),
                authority:
ctx.accounts.authority.to_account_info(),
            },
        ),
        collateral,
    )?;

    emit!(MemberJoined {
        circle: circle_key,
        member:
ctx.accounts.authority.key(),
        slot,

```

```

    });
    Ok(())
}

/// Pay this round's contribution. The
round's recipient never pays into their own
round.
pub fn contribute(ctx:
Context<Contribute>) -> Result<()> {
    let circle = &mut
ctx.accounts.circle;
    let member = &mut
ctx.accounts.member;
    require!(circle.status ==
CircleStatus::Active,
AdasheError::WrongStatus);
    require!(member.slot !=
circle.round,
AdasheError::RecipientDoesNotPay);
    require!(member.settled ==
circle.round, AdasheError::AlreadySettled);

    member.settled =
member.settled.checked_add(1).ok_or(AdasheE
rror::MathOverflow)?;
    circle.paid_count =
circle.paid_count.checked_add(1).ok_or(Adas
heError::MathOverflow)?;
    let amount = circle.contribution;
    let (circle_key, round) =
(circle.key(), circle.round);

    token::transfer(
        CpiContext::new(

```

```

ctx.accounts.token_program.to_account_info(
),
        Transfer {
            from:
ctx.accounts.authority_token.to_account_inf
o(),
            to:
ctx.accounts.vault.to_account_info(),
            authority:
ctx.accounts.authority.to_account_info(),
        },
    ),
    amount,
)?;

    emit!(Contributed {
        circle: circle_key,
        member:
ctx.accounts.authority.key(),
        round,
    });
    Ok(())
}

/// Permissionless: after the round
deadline, cover a missing contribution from
/// the defaulter's collateral. Tokens
already sit in the vault; this is
accounting only.
pub fn cover_default(ctx:
Context<CoverDefault>) -> Result<()> {
    let now =
Clock::get()?.unix_timestamp;

```

```

        let circle = &mut
ctx.accounts.circle;
        let member = &mut
ctx.accounts.member;
        require!(circle.status ==
CircleStatus::Active,
AdasheError::WrongStatus);
        require!(now >
circle.round_deadline,
AdasheError::RoundNotExpired);
        require!(member.slot !=
circle.round,
AdasheError::RecipientDoesNotPay);
        require!(member.settled ==
circle.round, AdasheError::AlreadySettled);

        member.collateral = member
            .collateral

.checked_sub(circle.contribution)

.ok_or(AdasheError::InsufficientCollateral)
?;
        member.settled =
member.settled.checked_add(1).ok_or(AdasheE
rror::MathOverflow)?;
        member.defaults =
member.defaults.saturating_add(1);
        circle.paid_count =
circle.paid_count.checked_add(1).ok_or(Adas
heError::MathOverflow)?;

        emit!(DefaultCovered {
            circle: circle.key(),

```

```

        member: member.authority,
        round: circle.round,
    });
    Ok(())
}

/// Permissionless: once every non-
recipient has paid or been covered, send
the pot
/// to the recipient's token account
and advance the round.
pub fn release_payout(ctx:
Context<ReleasePayout>) -> Result<()> {
    let now =
Clock::get()?.unix_timestamp;
    let circle = &mut
ctx.accounts.circle;
    require!(circle.status ==
CircleStatus::Active,
AdasheError::WrongStatus);
    require!(
        circle.paid_count ==
circle.max_members - 1,
        AdasheError::RoundIncomplete
    );

    let amount = circle.pot_size()?;
    let paid_round = circle.round;

    // Effects first.
    let recipient = &mut
ctx.accounts.recipient;
    recipient.settled =
paid_round.checked_add(1).ok_or(AdasheError

```

```

::MathOverflow)?;
        circle.round =
paid_round.checked_add(1).ok_or(AdasheError
::MathOverflow)?;
        circle.paid_count = 0;
        if circle.round ==
circle.max_members {
            circle.status =
CircleStatus::Completed;
        } else {
            circle.round_deadline = now

.checked_add(circle.period_secs)

.ok_or(AdasheError::MathOverflow)?;
        }

        // Interaction: vault -> recipient,
signed by the circle PDA.
        let circle_id =
circle.circle_id.to_le_bytes();
        let bump = [circle.bump];
        let seeds: &[&[u8]] = &[b"circle",
circle.creator.as_ref(), &circle_id,
&bump];
        let signer: &[&[&[u8]]] = &[seeds];
        token::transfer(
            CpiContext::new_with_signer(

ctx.accounts.token_program.to_account_info(
),
            Transfer {
                from:
ctx.accounts.vault.to_account_info(),

```

```

        to:
ctx.accounts.recipient_token.to_account_info(),
        authority:
circle.to_account_info(),
    },
    signer,
),
amount,
)?;

emit!(PayoutReleased {
    circle: circle.key(),
    recipient: recipient.authority,
    round: paid_round,
    amount,
});
Ok(())
}

/// After the circle completes, each
member withdraws whatever collateral
remains.
pub fn withdraw_collateral(ctx:
Context<WithdrawCollateral>) -> Result<()>
{
    let circle = &ctx.accounts.circle;
    require!(circle.status ==
CircleStatus::Completed,
AdasheError::WrongStatus);
    let member = &mut
ctx.accounts.member;
    let amount = member.collateral;
    require!(amount > 0,

```

```

AdasheError::NothingToWithdraw);
    member.collateral = 0;

    let circle_id =
circle.circle_id.to_le_bytes();
    let bump = [circle.bump];
    let seeds: &&[u8] = &[b"circle",
circle.creator.as_ref(), &circle_id,
&bump];
    let signer: &&[u8] = &[seeds];
    token::transfer(
        CpiContext::new_with_signer(

ctx.accounts.token_program.to_account_info(
),
        Transfer {
            from:
ctx.accounts.vault.to_account_info(),
            to:
ctx.accounts.authority_token.to_account_inf
o(),
            authority:
circle.to_account_info(),
        },
        signer,
    ),
    amount,
)?;

    emit!(CollateralWithdrawn {
        circle: circle.key(),
        member: member.authority,
        amount,
    });

```

```

        Ok(())
    }

    /// If the circle never filled before
    the join deadline, members reclaim
    collateral
    /// and close their member account
    (rent returns to them).
    pub fn refund_unfilled(ctx:
    Context<RefundUnfilled>) -> Result<()> {
        let now =
    Clock::get()?.unix_timestamp;
        let circle = &mut
    ctx.accounts.circle;
        require!(circle.status ==
    CircleStatus::Forming,
    AdasheError::WrongStatus);
        require!(now >
    circle.join_deadline,
    AdasheError::JoinWindowOpen);

        let amount =
    ctx.accounts.member.collateral;
        circle.member_count = circle
        .member_count
        .checked_sub(1)

    .ok_or(AdasheError::MathOverflow)?;

        let circle_id =
    circle.circle_id.to_le_bytes();
        let bump = [circle.bump];
        let seeds: &[&[u8]] = &[b"circle",
    circle.creator.as_ref(), &circle_id,

```

```

&bump];
    let signer: &[&[&[u8]]] = &[seeds];
    token::transfer(
        CpiContext::new_with_signer(

ctx.accounts.token_program.to_account_info(
),
        Transfer {
            from:
ctx.accounts.vault.to_account_info(),
            to:
ctx.accounts.authority_token.to_account_inf
o(),
            authority:
circle.to_account_info(),
        },
        signer,
    ),
    amount,
)?;

    emit!(CollateralWithdrawn {
        circle: circle.key(),
        member:
ctx.accounts.authority.key(),
        amount,
    });
    Ok(())
}

// _____ State
_____

```

```

#[derive(AnchorSerialize,
AnchorDeserialize, Clone, Copy, PartialEq,
Eq, InitSpace)]
pub enum CircleStatus {
    Forming,
    Active,
    Completed,
}

#[account]
#[derive(InitSpace)]
pub struct Circle {
    pub creator: Pubkey,
    pub mint: Pubkey,
    pub vault: Pubkey,
    pub circle_id: u64,
    pub contribution: u64,
    pub period_secs: i64,
    pub join_deadline: i64,
    pub round_deadline: i64,
    pub max_members: u8,
    pub member_count: u8,
    pub round: u8,
    pub paid_count: u8,
    pub status: CircleStatus,
    pub bump: u8,
}

impl Circle {
    /// Pot per round and per-member
    collateral: contribution * (N - 1).
    pub fn pot_size(&self) -> Result<u64> {
        self.contribution
            .checked_mul((self.max_members

```

```

- 1) as u64)
        .ok_or(error!
(AdasheError::MathOverflow))
    }
}

#[account]
#[derive(InitSpace)]
pub struct Member {
    pub circle: Pubkey,
    pub authority: Pubkey,
    pub slot: u8,
    /// Number of rounds this member has
settled (paid, covered, or been paid out).
    pub settled: u8,
    pub collateral: u64,
    pub defaults: u8,
    pub bump: u8,
}

// _____
Accounts _____

#[derive(Accounts)]
#[instruction(circle_id: u64)]
pub struct CreateCircle<'info> {
    #[account(mut)]
    pub creator: Signer<'info>,
    pub mint: Account<'info, Mint>,
    #[account(
        init,
        payer = creator,
        space = 8 + Circle::INIT_SPACE,
        seeds = [b"circle",

```

```

creator.key().as_ref(),
&circle_id.to_le_bytes()],
    bump
)]
pub circle: Account<'info, Circle>,
#[account(
    init,
    payer = creator,
    token::mint = mint,
    token::authority = circle,
    seeds = [b"vault",
circle.key().as_ref()],
    bump
)]
pub vault: Account<'info,
TokenAccount>,
pub token_program: Program<'info,
Token>,
pub system_program: Program<'info,
System>,
}

#[derive(Accounts)]
pub struct JoinCircle<'info> {
    #[account(mut)]
    pub authority: Signer<'info>,
    #[account(mut)]
    pub circle: Account<'info, Circle>,
    #[account(
        init,
        payer = authority,
        space = 8 + Member::INIT_SPACE,
        seeds = [b"member",
circle.key().as_ref()],

```

```

authority.key().as_ref()],
    bump
)]
pub member: Account<'info, Member>,
#[account(
    mut,
    token::mint = circle.mint,
    token::authority = authority
)]
pub authority_token: Account<'info,
TokenAccount>,
#[account(mut, address = circle.vault)]
pub vault: Account<'info,
TokenAccount>,
pub token_program: Program<'info,
Token>,
pub system_program: Program<'info,
System>,
}

#[derive(Accounts)]
pub struct Contribute<'info> {
    pub authority: Signer<'info>,
    #[account(mut)]
    pub circle: Account<'info, Circle>,
    #[account(
        mut,
        has_one = circle,
        seeds = [b"member",
circle.key().as_ref(),
authority.key().as_ref()],
        bump = member.bump
    )]
    pub member: Account<'info, Member>,

```

```

    #[account(
        mut,
        token::mint = circle.mint,
        token::authority = authority
    )]
    pub authority_token: Account<'info,
TokenAccount>,
    #[account(mut, address = circle.vault)]
    pub vault: Account<'info,
TokenAccount>,
    pub token_program: Program<'info,
Token>,
}

#[derive(Accounts)]
pub struct CoverDefault<'info> {
    #[account(mut)]
    pub circle: Account<'info, Circle>,
    #[account(mut, has_one = circle)]
    pub member: Account<'info, Member>,
}

#[derive(Accounts)]
pub struct ReleasePayout<'info> {
    #[account(mut)]
    pub circle: Account<'info, Circle>,
    #[account(
        mut,
        has_one = circle,
        constraint = recipient.slot ==
circle.round @ AdasheError::NotRecipient
    )]
    pub recipient: Account<'info, Member>,
    #[account(

```

```

        mut,
        token::mint = circle.mint,
        token::authority =
recipient.authority
    )]
    pub recipient_token: Account<'info,
TokenAccount>,
    #[account(mut, address = circle.vault)]
    pub vault: Account<'info,
TokenAccount>,
    pub token_program: Program<'info,
Token>,
}

#[derive(Accounts)]
pub struct WithdrawCollateral<'info> {
    pub authority: Signer<'info>,
    pub circle: Account<'info, Circle>,
    #[account(
        mut,
        has_one = circle,
        seeds = [b"member",
circle.key().as_ref(),
authority.key().as_ref()],
        bump = member.bump
    )]
    pub member: Account<'info, Member>,
    #[account(
        mut,
        token::mint = circle.mint,
        token::authority = authority
    )]
    pub authority_token: Account<'info,
TokenAccount>,

```

```

        #[account(mut, address = circle.vault)]
        pub vault: Account<'info,
TokenAccount>,
        pub token_program: Program<'info,
Token>,
    }

#[derive(Accounts)]
pub struct RefundUnfilled<'info> {
    #[account(mut)]
    pub authority: Signer<'info>,
    #[account(mut)]
    pub circle: Account<'info, Circle>,
    #[account(
        mut,
        close = authority,
        has_one = circle,
        seeds = [b"member",
circle.key().as_ref(),
authority.key().as_ref()],
        bump = member.bump
    )]
    pub member: Account<'info, Member>,
    #[account(
        mut,
        token::mint = circle.mint,
        token::authority = authority
    )]
    pub authority_token: Account<'info,
TokenAccount>,
    #[account(mut, address = circle.vault)]
    pub vault: Account<'info,
TokenAccount>,
    pub token_program: Program<'info,

```

```
Token>,
}

// _____
Events _____

#[event]
pub struct CircleCreated {
    pub circle: Pubkey,
    pub creator: Pubkey,
    pub mint: Pubkey,
    pub contribution: u64,
    pub max_members: u8,
    pub period_secs: i64,
}
#[event]
pub struct MemberJoined {
    pub circle: Pubkey,
    pub member: Pubkey,
    pub slot: u8,
}
#[event]
pub struct Contributed {
    pub circle: Pubkey,
    pub member: Pubkey,
    pub round: u8,
}
#[event]
pub struct DefaultCovered {
    pub circle: Pubkey,
    pub member: Pubkey,
    pub round: u8,
}
#[event]
```

```

pub struct PayoutReleased {
    pub circle: Pubkey,
    pub recipient: Pubkey,
    pub round: u8,
    pub amount: u64,
}

#[event]
pub struct CollateralWithdrawn {
    pub circle: Pubkey,
    pub member: Pubkey,
    pub amount: u64,
}

// _____
Errors _____

#[error_code]
pub enum AdasheError {
    #[msg("Invalid circle parameters")]
    InvalidParams,
    #[msg("Arithmetic overflow")]
    MathOverflow,
    #[msg("Circle is not in the right
status for this action")]
    WrongStatus,
    #[msg("Join window has closed")]
    JoinWindowClosed,
    #[msg("Join window is still open")]
    JoinWindowOpen,
    #[msg("Circle is full")]
    CircleFull,
    #[msg("The recipient does not pay into
their own round")]
    RecipientDoesNotPay,
}

```

```
        #[msg("Member already settled this
round")]
        AlreadySettled,
        #[msg("Round deadline has not passed")]
        RoundNotExpired,
        #[msg("Member collateral is
insufficient")]
        InsufficientCollateral,
        #[msg("Round is not fully funded yet")]
        RoundIncomplete,
        #[msg("Provided member is not this
round's recipient")]
        NotRecipient,
        #[msg("No collateral left to
withdraw")]
        NothingToWithdraw,
    }
```